CHI Interconnect Verification Environment

Verification Specification

CHI RTL project

Baseline: Arm IHI 0050H, Issue H

Contents

Document status

1 Scope and verification status

- 1.1 What is verified
- 1.2 Status definitions
- 1.3 Verification feature matrix
- 1.4 Scope of Stage 1

2 Environment architecture

- 2.1 Architecture at a glance
- 2.2 Component inventory
- 2.3 The configuration database contract
- 2.4 Node counts, agent indices and node ordinals
- 2.5 Configuration objects

3 Global definitions

- 3.1 Derived parameters
- 3.2 Types and enumerations
- 3.3 Helper functions

4 The node interface and the link layer

- 4.1 One interface, and why the DUT's own is reused
- 4.2 Signal ownership
- 4.3 Link activation
- 4.4 L-Credit accounting

5 Requester agent

- 5.1 Structure
- 5.2 The sequence item
- 5.3 The cache model
- 5.4 Snoop response decision
- 5.5 Driver thread structure
- 5.6 TxnID allocation
- 5.7 Access flows
- 5.8 Retry and P-Credit flow
- 5.9 Monitor

6 Subordinate agent

- 6.1 Structure and policy
- 6.2 Memory model
- 6.3 Request service
- 6.4 Monitor

7 Checking

- 7.1 Reference models
- 7.2 Check inventory
- 7.3 End-of-test checks

8 Coverage plan
8.1 Cross-agent coverage
8.2 Requester coverage
8.3 Subordinate coverage
8.4 Known coverage holes
9 Stimulus
9.1 Requester sequences
9.2 Subordinate policy sequences
9.3 Virtual sequences
10 Test plan
10.1 Test list
10.2 Base test
10.3 Regression status
11 Run flow
11.1 Directory map
11.2 Compile order
11.3 Make targets
11.4 Waveforms
11.5 Regenerating this document
12 Traceability
13 Known limitations

Document status

This document specifies the UVM verification environment in `DV/` that drives the CHI interconnect RTL in `rtl/`. It describes what the environment builds, what it drives, what it checks and what it covers, at the level of detail needed to review the environment without reading it.

Every statement below was checked against the source. Where the environment deliberately stops short of the architecture, the boundary is stated rather than implied: Section 1.4 lists what Stage 1 excludes and Section 13 lists what is known to be missing.

The companion document `docs/chi_interconnect_design_spec.docx`, generated from `doc-src/chi_interconnect_design_spec.md`, specifies the RTL. Where the two disagree about intended behaviour, the design specification describes the design and this document describes what is actually checked.

This document is generated from `DV/doc/chi_dv_specification.md`. Section 11.5 gives the regeneration command.

1 Scope and verification status

1.1 What is verified

The device under test is `chi_interconnect_top`, instantiated once with the default configuration. The environment attaches a protocol-level agent to every external node port of the fabric and nothing else: there is no white-box access, no backdoor into fabric state, and no reliance on internal signals. The fabric is exercised and judged entirely through `chi_if`.

The environment answers three questions:

1. **Does the link layer come up and stay legal?** Every channel in both directions must reach RUN, and no transmitter may ever hold more L-Credits than the channel depth.
2. **Does every transaction complete, exactly once, with a legal response?** Requests, retries, credit grants, snoops, data beats and completions are tracked per Requester and per TxnID, and anything left open at the end of the test is a failure.
3. **Is the data coherent?** Read data is compared against an independent memory model, and the directory model asserts that no line is held Unique by one Requester while another holds a copy.

1.2 Status definitions

Checked — the environment produces the stimulus and a checker fails the test when the behaviour is wrong.

Driven — the environment produces the stimulus and the response is accepted, but no independent model judges it beyond the generic transaction lifecycle and link-legality checks.

Not exercised — no stimulus reaches this behaviour. The fabric may implement it.

1.3 Verification feature matrix

Area	Issue H reference	Status	Boundary
Link activation to RUN, both directions, every channel	B14.5	Checked	`check_phase` fails if any of the four link state trackers per agent never observed RUN. Deactivation is never requested, so the DEACTIVATE path is not exercised.
L-Credit accounting	B13.11	Checked	The monitor tracks credits held as granted minus spent and the scoreboard fails if the balance exceeds `lcrdDepth`. Credit underflow is prevented by construction in the BFM rather than checked.
`LCrdReturn`	B13.11	Not exercised	The BFM never returns credits, because Stage 1 never deactivates a link.
System coherency handshake	B14.7	Driven	A fully coherent Requester asserts `SYSCOREQ` during bring-up and the monitor records `SYSCOACK`. No sequencing rule is checked.
Flit parity	B9.3	Not exercised	`parityEn` is left at its default. The BFM computes parity on transmit; no error is injected.
Coherent read, `ReadShared`	B4, B5.1	Checked	Issued on a cache miss. Data is compared byte for byte against the memory model and the granted state is recorded in the directory model.
Coherent read to Unique, `ReadUnique`	B5.1	Checked	Issued on a write miss, followed by a local store.
Upgrade, `CleanUnique`	B5.1	Checked	Issued on a write that hits a Shared line. Completion must be a bare `Comp`.
`ReadOnce`	B5.1	Checked	The only read an I/O coherent Requester issues. Data is checked; the Requester is not recorded as a holder.
Writeback, `WriteBackFull`	B5.3	Checked	Issued when a dirty line is evicted. The written line updates the memory model.
`Evict`	B5.3	Checked	Issued when a clean line is evicted. Completion must be a bare `Comp`.
`WriteUniqueFull`	B5.3	Checked	The only write an I/O coherent Requester issues.
Snoops and snoop responses	B4.3--B4.9	Checked	The cache model decides the response from the snoop opcode and the current state, and the scoreboard checks that a snooped Requester was recorded as a holder and that a non-cacheable Requester is never snooped.
Snoop-supplied data	B4.9	Checked	`SnpRespData` carries the line when the snoop returns data, and the scoreboard folds it into the memory model.
Request Retry, `RetryAck` and `PCrdGrant`	B2.10	Checked	Every retried request must resend with `AllowRetry` clear and the granted `PCrdType`, and every `RetryAck` must be followed by a grant before the test ends.
`PCrdReturn`	B2.10.1	Not exercised	A granted credit is always spent, never returned.
Transaction completion and `CompAck`	B2.3.2	Checked	Completion is recorded once per transaction; a second completion is an error.

Directory and sharer invariants	B4.4	Checked	Unique exclusivity is checked on every completion that names a cache state.
Read data integrity	—	Checked	Against the memory model, skipped for a line with a concurrent write in flight.
Downstream memory traffic	B5.3	Checked	The Subordinate model serves reads from its own memory and applies writes, and the scoreboard mirrors both.
Address map, HN-F versus HN-I regions	B3.1	Checked	An SN-F that receives an address the map routes to the HN-I region is an error.
Error responses	B2.8	Driven	The Subordinate has an injection hook, left at `RESP_ERR_OK`. Any `DERR` or `NDERR` observed with no injection is an error.
DMT, DWT, DCT	B2.5.1	Not exercised	Disabled in `chiGlobalsPkg` so the Home stays in the data path (Section 1.4).
Atomics, CMOs, exclusives, stash, DVM	B6, B7, B8	Not exercised	No sequence issues them.
Partial writes and byte enables	B5.3	Not exercised	Every access is a full line, `Size` is fixed at 6.
MultiReq, PBHA, MTE, MEC, MPAM	B2.8, B11, B12	Not exercised	Carried by the flit records, never set.

Table 1.1 Verification status by area

1.4 Scope of Stage 1

Stage 1 includes:

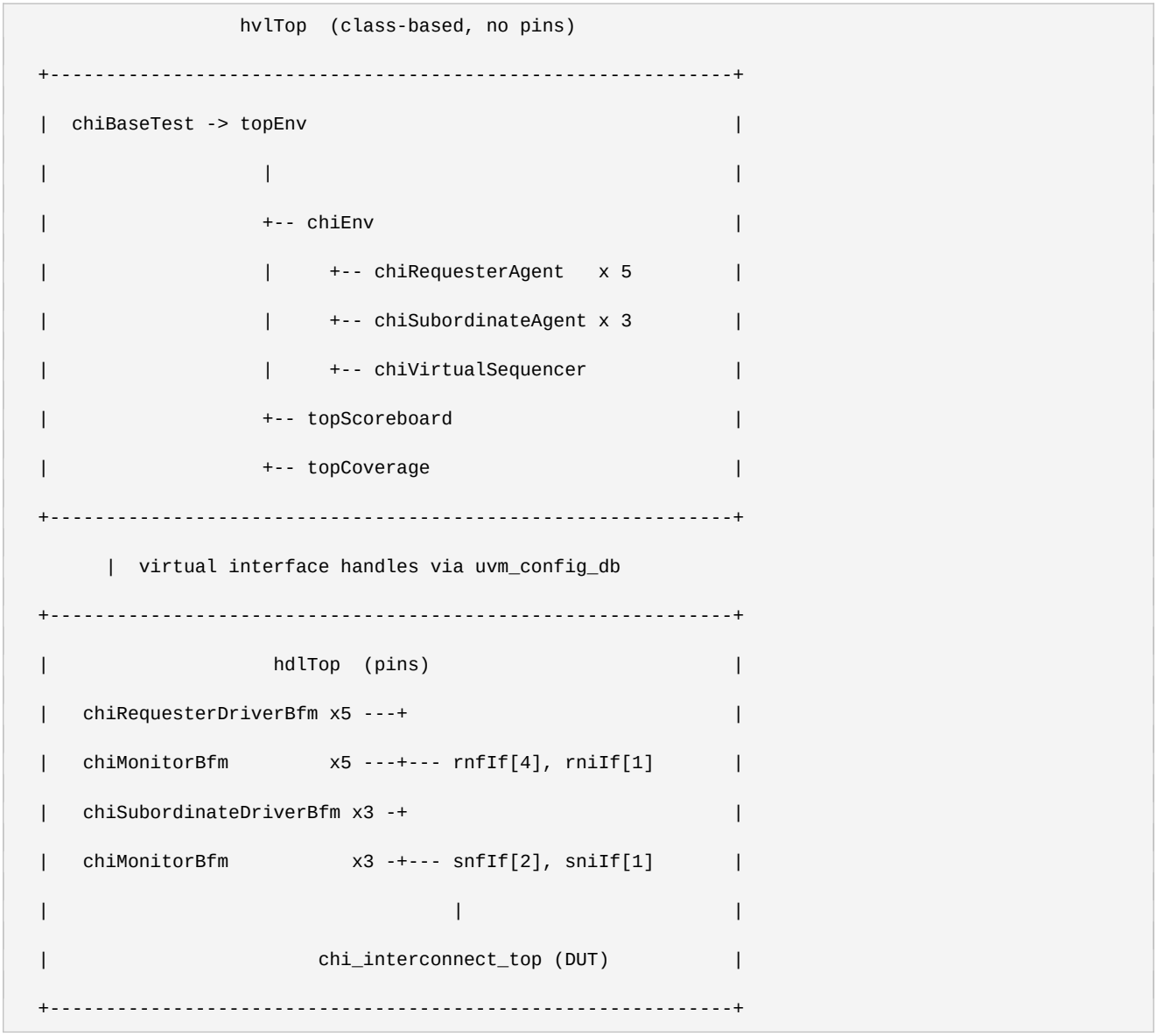
- Link bring-up on all eight external ports and both directions of each.
- Coherent read and write traffic from four RN-F Requesters, with snoops arising from the traffic rather than being injected.
- I/O coherent read and write traffic from one RN-I Requester.
- Downstream memory service from two SN-F and one SN-I Subordinate.
- The full Request Retry flow, including credited resends.
- Cache-capacity eviction, both clean and dirty.
- Checking against independent address-map, directory and memory models.
- Functional coverage of opcodes, cache states, link states and retry.

Stage 1 does not include:

- DMT, DWT and DCT. `chiGlobalsPkg` clears `dmtEn`, `dwtEn` and `dctEn` unless `CHI_TB_ENABLE_DIRECT` is defined, so the Home remains in the data path and the scoreboard sees every beat. Enabling them changes who sources `CompData` and who owns the DBID, and needs its own checker work.
- Atomics, CMOs, exclusives, cache stashing and DVM.
- Partial-line and unaligned accesses.
- Error injection and error-response checking.
- Link deactivation and credit return.
- Any configuration other than the default node counts.

2 Environment architecture

2.1 Architecture at a glance



The split is the conventional one. `hdlTop` holds everything that touches a pin: the DUT, the `chi_if` arrays, the driver and monitor BFMs, and the clock and reset generator. `hvlTop` holds the class-based environment and never sees a signal; it reaches the pins only through the virtual interface handles that `hdlTop` publishes in the configuration database.

2.2 Component inventory

Path	Component	Role
`src/globals/chiGlobalsPkg.sv`	`chiGlobalsPkg`	Configuration, derived counts, TB types and helper functions
`src/hdlTop/hdlTop.sv`	`hdlTop`	DUT instance, `chi_if` arrays, BFM

		generation, config-db publication
`src/hdlTop/chiInterface/ chiClkRstBfm.sv`	`chiClkRstBfm`	Clock and reset generation, cycle waiting
`src/hdlTop/chiLink/ chiLinkLayerBfm.svh`	(include)	Shared link layer: activation, credits, flit queues
`src/hdlTop/chiRequesterAgentBfm/ chiRequesterDriverBfm.sv`	`chiRequesterDriverBfm`	Requester-side pin driving and flit reception
`src/hdlTop/chiSubordinateAgentBfm/ chiSubordinateDriverBfm.sv`	`chiSubordinateDriverBfm`	Subordinate-side pin driving and flit reception
`src/hdlTop/chiMonitor/ chiMonitorBfm.sv`	`chiMonitorBfm`	Passive flit and link-event sampling for either role
`src/hvlTop/chiEnv/chiRequester/`	`chiRequesterAgent` and parts	Requester agent: config, item, cache model, driver proxy, monitor proxy, coverage, sequencer, sequences
`src/hvlTop/chiEnv/chiSubordinate/`	`chiSubordinateAgent` and parts	Subordinate agent: config, item, memory model, driver proxy, monitor proxy, coverage, sequencer, sequences
`src/hvlTop/chiEnv/ chiVirtualSequencer/`	`chiVirtualSequencer`	Handles to every agent sequencer
`src/hvlTop/chiEnv/ chiVirtualSequences/`	virtual sequences	Whole-system stimulus
`src/hvlTop/chiEnv/chiEnv.sv`	`chiEnv`	Builds and connects the agents and the virtual sequencer
`src/hvlTop/topEnv/topScoreboard.sv`	`topScoreboard`	All checking and the reference models
`src/hvlTop/topEnv/topCoverage.sv`	`topCoverage`	Cross-agent functional coverage
`src/hvlTop/topEnv/topEnv.sv`	`topEnv`	Wraps `chiEnv`, scoreboard and coverage, and fans out the analysis ports
`src/hvlTop/test/`	`chiBaseTest`, tests	Test library
`src/hvlTop/hvlTop.sv`	`hvlTop`	`run_test()`

Table 2.1 Component inventory

2.3 The configuration database contract

`hdlTop` publishes one handle per BFM, each under a key that carries the agent index, and one handle for the clock and reset BFM. All are set on the null context with a wildcard scope.

Key	Type	Consumer
`chiRequesterDriverBfm_%0d`	`virtual chiRequesterDriverBfm`	`chiRequesterDriverProxy`

chiRequesterMonitorBfm_%0d	virtual chiMonitorBfm	chiRequesterMonitorProxy
chiSubordinateDriverBfm_%0d	virtual chiSubordinateDriverBfm	chiSubordinateDriverProxy
chiSubordinateMonitorBfm_%0d	virtual chiMonitorBfm	chiSubordinateMonitorProxy
chiClkRstBfm	virtual chiClkRstBfm	chiBaseTest

Table 2.2 Virtual interface handles

A passive agent builds its monitor proxy and no driver proxy, so it fetches only the monitor handle. This is what `chiPassiveAgentTest` exercises.

2.4 Node counts, agent indices and node ordinals

The DUT places every node in a single ordinal space, and the NodeID of a node is a function of its ordinal. The environment therefore cannot number its agents freely: agent index, node class, node ordinal and NodeID are all tied together by `chiGlobalsPkg`.

With the default configuration the space is:

Ordinal	Class	Agent	NodeID
0--3	RN-F	requester 0--3	`0x40`, `0xc1`, `0x242`, `0x343`
4	RN-I	requester 4	`0x144`
5--6	SN-F	subordinate 0--1	`0x85`, `0x306`
7	SN-I	subordinate 2	`0x207`
8--11	HN-F	not an agent	—
12	HN-I	not an agent	—
13	MN	not an agent	—

Table 2.3 Node ordinal space in the default configuration

`numRnd` is zero by default, so there is no RN-D agent and requester index 4 is the RN-I. The `rndIf` array is still declared with one element, because a zero-length interface array cannot be elaborated; nothing drives it.

Five requester agents and three subordinate agents attach to the eight external ports. The four Home Nodes, the HN-I and the MN are internal to the fabric and have no agent: they are exercised only through the traffic that reaches them.

2.5 Configuration objects

`topEnvConfig` holds `hasScoreboard` and `hasCoverage`, both set by default, and a `chiEnvConfig`. `chiEnvConfig`

holds `numRequesterAgents`, `numSubordinateAgents` and an array of per-agent configuration objects, which it fills in `initialise()` from the derived counts.

Field	Type	Default
`agentId`	`int unsigned`	set by `chiEnvConfig`
`nodeClass`	`chi_node_class_e`	`requesterClassOf(agentId)`
`nodeOrdinal`	`int unsigned`	`requesterOrdinalOf(agentId)`
`nodeId`	`chiNodeIdT`	`nodeIdOf(nodeOrdinal)`
`isActive`	`uvm_active_passive_enum`	`UVM_ACTIVE`
`hasCache`	`bit`	`isFullyCoherent(nodeClass)`
`usesSystemCoherency`	`bit`	`isFullyCoherent(nodeClass)`
`maxOutstanding`	`int unsigned`	4
`allowRetry`	`bit`	1

Table 2.4 `chiRequesterAgentConfig`

Field	Type	Default
`agentId`	`int unsigned`	set by `chiEnvConfig`
`nodeClass`	`chi_node_class_e`	`subordinateClassOf(agentId)`
`nodeOrdinal`	`int unsigned`	`subordinateOrdinalOf(agentId)`
`nodeId`	`chiNodeIdT`	`nodeIdOf(nodeOrdinal)`
`isActive`	`uvm_active_passive_enum`	`UVM_ACTIVE`
`maxConcurrentRequests`	`int unsigned`	8
`defaultResponseDelay`	`int unsigned`	2

Table 2.5 `chiSubordinateAgentConfig`

`hasCache` is the single switch that decides whether a Requester behaves as a fully coherent node with a cache model and a snoop responder, or as an I/O coherent node that holds nothing. `maxOutstanding` is what decides whether the retry path is ever reached: with one transaction per Requester the fabric never runs short of resources and no `RetryAck` is ever sent.

3 Global definitions

3.1 Derived parameters

`chiGlobalsPkg` reuses the DUT's `chi_cfg_pkg` and `chi_pkg` rather than restating them. `CHI_CFG` is built from `CHI_CFG_DEFAULT` by `buildChiCfg()`, which applies the compile-time overrides in Table 3.2 and, unless `CHI_TB_ENABLE_DIRECT` is defined, clears `dmtEn`, `dwtEn` and `dctEn`.

Parameter	Derivation	Default value
`NUM_NODES`	sum of all eight node counts	14
`OFF_RNF`, `OFF_RND`, `OFF_RNI`	running sum of the counts	0, 4, 4
`OFF_SNF`, `OFF_SNI`	running sum of the counts	5, 7
`OFF_HNF`, `OFF_HNI`, `OFF_MN`	running sum of the counts	8, 12, 13
`NUM_REQUESTER_AGENTS`	`numRnf + numRnd + numRni`	5
`NUM_SUBORDINATE_AGENTS`	`numSnf + numSni`	3
`NUM_EXTERNAL_PORTS`	requester plus subordinate agents	8
`NUM_RNF_IF` and friends	count, floored at 1	4, 1, 1, 2, 1
`CHI_ADDR_W`, `CHI_DATA_W`, `CHI_NODEID_W`	`CHI_CFG` fields	48, 256, 11
`REQ_FLIT_W`, `RSP_FLIT_W`, `SNP_FLIT_W`, `DAT_FLIT_W`	`chi_pkg` width functions	153, 79, 106, 437
`CHI_LINE_BYTES`, `CHI_LINE_OFF_W`	fixed by the 64-byte line	64, 6
`CHI_LINE_ADDR_W`	`CHI_ADDR_W - CHI_LINE_OFF_W`	42
`CHI_DAT_PKTS_PER_LINE`	`CHI_LINE_BYTES * 8 / CHI_DATA_W`	2
`CHI_WIDEST_FLIT_W`	max of the four architectural maxima	see below

Table 3.1 Derived parameters

`CHI_WIDEST_FLIT_W` is the one parameter that must not be derived from this configuration's flit widths. `chi_if` carries the *architectural* maximum on every channel, so a monitor that part-selects a flit using this configuration's `DAT_FLIT_W` reads off the end of the payload and pulls in X. It is therefore `chiMaxOf4` of `CHI_MAX_REQ_FLIT_W`, `CHI_MAX_RSP_FLIT_W`, `CHI_MAX_SNP_FLIT_W` and `CHI_MAX_DAT_FLIT_W`. This was a real bug; see `iteration-log.md` item 8.

Define	Effect
`CHI_TB_NUM_RNF`, `CHI_TB_NUM_RND`, `CHI_TB_NUM_RNI`	Override the Requester counts

`CHI_TB_NUM_SNF`, `CHI_TB_NUM_SNI`	Override the Subordinate counts
`CHI_TB_LCRD_DEPTH`	Override the channel credit depth
`CHI_TB_PARITY_EN`	Set `parityEn`
`CHI_TB_ENABLE_DIRECT`	Leave `dmtEn`, `dwtEn` and `dctEn` as configured

Table 3.2 Compile-time overrides

Raising the node counts past the defaults is not supported: the DUT's placement table enumerates ordinals 0 to 13 only. See `open-questions.md` item 3.

3.2 Types and enumerations

Type	Definition	Purpose
`chiAddrT`	`logic [CHI_ADDR_W-1:0]`	Byte address
`chiLineAddrT`	`logic [CHI_LINE_ADDR_W-1:0]`	Cache line address, the key of every line-keyed model
`chiDataT`	`logic [CHI_DATA_W-1:0]`	One data beat
`chiLineBytesT`	`chiByteT [CHI_LINE_BYTES]`	A whole line as bytes, which is how both memory models and every data check work
`chiNodeIdT`, `chiTxnIdT`	widths from `chi_pkg`	Identity fields at their flit slot width
`chiTxnKeyT`	packed `{srcId, txnId}`	The pair that makes a transaction unique
`chiSampledFlitS`	`{direction, channel, flit}`	What a monitor BFM hands up
`chiLinkEventS`	`{kind, direction, channel, state, creditCount, sysCoReq, sysCoAck}`	Link activity as a transaction
`chiSnoopActionS`	`{nextState, respState, sendData, passDirty}`	The cache model's decision for one snoop

Table 3.3 Testbench types

Enumeration	Members
`chiDirectionE`	`CHI_DIR_TX`, `CHI_DIR_RX`
`chiLinkStateE`	`CHI_LINK_STOP`, `CHI_LINK_ACTIVATE`, `CHI_LINK_RUN`, `CHI_LINK_DEACTIVATE`
`chiTxKindE`	`CHI_TX_PROTOCOL_FLIT`, `CHI_TX_LINK_EVENT`
`chiLinkEventKindE`	`CHI_LINK_EVT_STATE_CHANGE`, `CHI_LINK_EVT_CREDIT_GRANT`, `CHI_LINK_EVT_CREDIT_USE`, `CHI_LINK_EVT_CREDIT_ERROR`, `CHI_LINK_EVT_SYSCO`
`chiAccessKindE`	`CHI_ACCESS_READ`, `CHI_ACCESS_WRITE`, `CHI_ACCESS_READ_UNIQUE`, `CHI_ACCESS_CLEAN`,

	`CHI_ACCESS_EVICT`

Table 3.4 Testbench enumerations

`chiTxKindE` is what lets one sequence item and one analysis port carry both protocol flits and link events, so the scoreboard receives link activity in the same stream and in the same order as the traffic it constrains.

Everything else is the DUT's: `chi_node_class_e`, `chi_channel_e`, `chi_cache_state_e`, `chi_resp_e`, the opcode enumerations and the four flit records all come from `chi_pkg`. The environment deliberately does not restate them, so a flit field can never drift between the RTL and the checker.

3.3 Helper functions

Function	Purpose
`isRequesterClass`, `isSubordinateClass`	Which side of the fabric a class sits on
`isFullyCoherent`, `isIoCoherent`	Whether a Requester has a cache and is snooped
`hasDvmSupport`	RN-D with `dvmEn`, for future DVM work
`requesterClassOf`, `subordinateClassOf`	Agent index to node class
`requesterOrdinalOf`, `subordinateOrdinalOf`	Agent index to node ordinal
`requesterClassIndexOf`, `subordinateClassIndexOf`	Agent index to index within its `chi_if` array
`nodeIdOf`	Node ordinal to NodeID, by way of the DUT's `defaultNodeId`
`channelApplies`	Whether a class uses a channel in a direction
`lineAddrOf`, `lineBaseOf`	Address to line address and to line base
`memInitByte`	The contents of an unwritten byte
`chiMaxOf4`	Maximum of four unsigned integers

Table 3.5 Helper functions

`memInitByte(addr)` returns `addr[7:0] ^ addr[15:8] ^ addr[23:16] ^ 8'hA5`. Both the Subordinate's memory and the scoreboard's memory model use it, so an unwritten line has known, address-dependent, non-zero contents and a read that returns zeros or X fails immediately instead of matching by accident.

4 The node interface and the link layer

4.1 One interface, and why the DUT's own is reused

The DV environment declares no interface of its own. `rtl/if/chi_if.sv` is the only CHI interface in the repository, it is a superset covering all five external node classes, and every DUT port is an instance of it. Declaring a

second one in the testbench would create exactly the drift the DUT avoided by having one. The driver and monitor BFMs are therefore written as interfaces that take a `chi_if` port.

4.2 Signal ownership

The fabric's own link ports drive more of `chi_if` than the specification assigns to the fabric, which constrains what the BFM may drive: anything the fabric drives would otherwise have two drivers.

Driven by the node-side BFM	Driven by the fabric
`TXLINKACTIVEREQ`	`CLK`, `RESETn`
`RXLINKACTIVEACK`	`TXLINKACTIVEACK`, `RXLINKACTIVEREQ`
`SYSCOREQ`	`SYSCOACK`
RX L-Credit valids	TX L-Credit valids
TX flit groups and their parity	The whole RX flit group
	Every single-bit check pin
	`TXSACTIVE`, `RXSACTIVE`

Table 4.1 `chi_if` signal ownership

`TXSACTIVE` is the transmitting node's output in the specification but the fabric's output here, so an agent cannot signal activity independently of the fabric. This is recorded as `open-questions.md` item 1.

4.3 Link activation

Each side initiates the direction it transmits on and acknowledges the direction it receives on. The BFM raises `TXLINKACTIVEREQ`, waits for `TXLINKACTIVEACK`, and separately registers `RXLINKACTIVEREQ` into `RXLINKACTIVEACK`. Both directions are tracked as a four-state FSM so the monitor can report STOP, ACTIVATE, RUN and DEACTIVATE as link events rather than as raw pin changes.

System coherency entry is separate from link bring-up: `SYSCOREQ` is asserted only by a Requester whose `usesSystemCoherency` is set, because entry is a protocol-level decision rather than part of activation.

Stage 1 never deactivates a link. The acknowledgement therefore simply follows the request, and the DEACTIVATE state is reachable in the tracker but never reached in a test.

4.4 L-Credit accounting

The transmitter may present a flit only while it holds a credit for that channel. Because credits are only issued from RUN, holding a credit is itself proof that the far side is ready, which is what the DUT's own receive assertions `NO_RECEIVE_OUTSIDE_ACTIVE`, `NO_BUFFER_OVERFLOW` and `PEER_CREDITS_IN_RANGE` require. The BFM never presents a flit without a credit, so those three assertions are satisfied by construction rather than by being negotiated

at run time.

The receive side issues credits up to `LCRD_DEPTH`, which is `lcrdDepth` floored at 2.

The monitor tracks the credits *held* on each channel and direction as grants minus flits sent, and publishes the balance with every grant. Tracking cumulative grants instead was the first bug the environment found in itself: see `iteration-log.md` item 6.

5 Requester agent

5.1 Structure

A Requester agent is the standard proxy arrangement: a sequencer and a driver proxy on the active path, a monitor proxy and a coverage collector on the passive path, and a cache model shared between the driver proxy and nothing else. The driver proxy owns the protocol; the BFM owns the pins.

An I/O coherent Requester builds no cache model, answers no snoops and takes a different access path (Section 5.7).

5.2 The sequence item

`chiRequesterTx` carries an *intent*, not a flit. A sequence asks for a read or a write at an address; which CHI request that becomes is decided by the driver proxy against the cache state at the moment of issue, because that decision can only be made once and only there.

Field group	Fields
Intent	`accessKind`, `addr`, `size`, `qos`, `writeData`
Kind	`txKind`, `direction`, `channel`
Flits	`reqFlit`, `rspFlit`, `snpFlit`, `datFlit`
Link	`linkEvent`
Identity	`agentId`, `nodeClass`
Outcome	`completionResp`, `completionErr`, `retryCount`, `readData`
Validity	`addrValid`

Table 5.1 `chiRequesterTx` fields

Constraint	Content
`addressSpace`	`addr[CHI_ADDR_W-1] == 0` and the line offset is zero
`transferSize`	`size == 6`, a full 64-byte line

`intentKind`	A protocol flit, transmitted, on REQ
`qosRange`	`qos inside {[0:15]}`

Table 5.2 `chiRequesterTx` constraints

`addrValid` exists because RSP and DAT flits carry no address. The monitor sets it only for REQ and SNP, and the scoreboard and coverage resolve the address of a response from the transaction table instead of trusting the field. Copying the address field unconditionally was the cause of `iteration-log.md` item 7.

5.3 The cache model

The model is a bounded, fully associative line cache: 32 lines, each holding a state, the line bytes, a use stamp and an in-flight mark.

State	Meaning
`CACHE_I`	Not held
`CACHE_SC`	Shared clean
`CACHE_UC`	Unique clean
`CACHE_UD`	Unique dirty
`CACHE_UDP`	Unique dirty partial
`CACHE_SD`	Shared dirty

Table 5.3 Modelled cache states

Capacity and victim selection. When the cache is full the driver evicts before it allocates. The victim is the least recently used resident line that has no transaction in flight; if every resident line is busy, no victim is returned and the driver waits rather than evicting something it is using.

Residency and the in-flight mark are independent. A line is allocated and invalidated in the middle of the transaction that holds its mark, and it is invalidated by snoops while unrelated transactions are in flight, so neither allocation nor invalidation may clear the mark. Only the driver thread that took the mark releases it. Getting this wrong let two transactions run on one line: `iteration-log.md` item 13.

5.4 Snoop response decision

`snoopAction(addr, opcode, retToSrc)` returns the next state, the response state, whether data is returned and whether dirtiness is passed. The response state is derived from the next state, with the pass-dirty bit set on top.

Snoop opcode	Next state	Data returned
`SnpOnce`	unchanged	when `RetToSrc` and the line is valid
`SnpShared`	`SC` from a clean state, `SD` from a dirty one	when dirty, or when `RetToSrc`

`SnpNotSharedDirty`	`SC`	when dirty, passing dirty, or when `RetToSrc`
`SnpClean`, `SnpCleanShared`	`SC`	when dirty, passing dirty
`SnpUnique`, `SnpCleanInvalid`	`I`	when dirty, passing dirty; otherwise when `RetToSrc` and valid
`SnpMakeInvalid`	`I`	never
default	unchanged	when `RetToSrc` and the line is valid

Table 5.4 Snoop action table

State to response mapping: `I` to `RESP_I`, `SC` to `RESP_SC`, `UC`, `UD` and `UDP` to `RESP_UC_UD`, `SD` to `RESP_SD`, with bit 2 set when dirtiness is passed.

Snoop atomicity. A real cache keeps a line locked while it is being snooped. The handler here reads the line out, spends several cycles sending the data beats and only then applies the new state, so a store landing in that window would leave the snoop carrying pre-store bytes while this Requester kept post-store bytes. Stores and snoop processing therefore take a common lock: `iteration-log.md` item 14.

5.5 Driver thread structure

`run_phase` brings the link up and then forks four threads that run for the rest of the test:

Thread	Responsibility
`rspRouter`	Reads RSP flits and posts each into the mailbox of its TxnID, or into the P-Credit pool for a `PCrdGrant`
`datRouter`	Reads DAT flits and posts each into the mailbox of its TxnID
`snoopHandler`	Answers snoops, fully coherent Requesters only
`mainLoop`	Takes sequence items and launches an access thread for each

Table 5.5 Driver threads

`mainLoop` hands the item back as soon as the access has been *launched*, not when it completes, and limits concurrency with a semaphore of `maxOutstanding`. Holding each item to completion would make every agent single-threaded, the Home would never run short of resources, and the retry path would never be reached.

Because several access threads run concurrently within one agent, the same-line rule has to be enforced explicitly. An access claims its line before it does anything that can block, and the check and the claim happen in one step: any wait between them lets a second thread find the line free too (`iteration-log.md` item 12).

5.6 TxnID allocation

A TxnID is unique only among the transactions of one source node, so the pool covers one agent. Allocation scans candidates 0 to 63 under a lock, takes the first free value, and creates the RSP and DAT mailboxes for it. The ID is released when the transaction completes.

5.7 Access flows

Access	Cache state at issue	Request issued	Completion
Read	any valid state	none, hit	—
Read	invalid	`ReadShared`, `ExpCompAck` set	`CompData` beats, then `CompAck`
Write	Unique	none, silent store	—
Write	Shared	`CleanUnique`, `ExpCompAck` set	`Comp`, then `CompAck`, then store
Write	invalid	`ReadUnique`, `ExpCompAck` set	`CompData` beats, then `CompAck`, then store
Evict	dirty	`WriteBackFull`	`DBIDResp`, `CopyBackWrData` beats, `Comp`
Evict	clean	`Evict`	`Comp`
I/O read	no cache	`ReadOnce`	`CompData` beats
I/O write	no cache	`WriteUniqueFull`	`DBIDResp`, `NonCopyBackWrData` beats, `Comp`

Table 5.6 Access to request mapping

Writes store a pattern derived from the line base address, the agent index and a per-agent serial number, so the bytes in any line identify who wrote them and when, and every later reader can be checked against them.

Read completion accepts `CompData` and `DataSepResp`. The driver raises an error if a read is answered by anything else, if a `CleanUnique` or `Evict` is answered by anything but `Comp`, or if a response or data flit arrives for a TxnID that is not outstanding.

5.8 Retry and P-Credit flow

`issueWithRetry` implements the full flow:

1. Send the request with `AllowRetry` set.
2. Watch the RSP and DAT mailboxes together. An accepted read answers on DAT and never says anything on RSP, so watching RSP alone deadlocks (`iteration-log.md` item 9).
3. If the answer is `RetryAck`, wait for a `PCrdGrant` from that Home for that credit type, then resend the same request with `AllowRetry` clear, the granted `PCrdType`, and the granting Home as `TgtID`.
4. Anything else means the request was accepted.

P-Credits are pooled per `(Home NodeID, PCrdType)` pair, because a grant names a credit type at a Home and is not tied to the request that was retried.

5.9 Monitor

The monitor proxy reads flits and link events from the monitor BFM and publishes both as `chiRequesterTx` on one analysis port, discriminated by `txKind`. It unpacks each flit into the matching record, sets `addr` and `addrValid` for

REQ and SNP only, and tags every item with the agent index and node class. It is built whether the agent is active or passive.

6 Subordinate agent

6.1 Structure and policy

The Subordinate is reactive: it has no stimulus of its own. Its sequencer carries *policy* items rather than traffic, giving a test control over response latency and error injection. When no policy item is available the driver uses a default built from `defaultResponseDelay` with no error, so a Subordinate always answers even if no sequence runs on it.

6.2 Memory model

`chiSubordinateMemoryModel` is a sparse byte-addressed memory. A byte that has never been written reads as `memInitByte(addr)`, so memory is deterministically initialised without being materialised. It supports byte and line access and a masked line write for partial writes.

6.3 Request service

Request class	Opcodes	Response
Read	`ReadNoSnp`, `ReadNoSnpSep`, `ReadOnce`, `ReadShared`, `ReadClean`, `ReadUnique`, `ReadNotSharedDirty`, `ReadOnceCleanInvalid`, `ReadOnceMakeInvalid`, `ReadPreferUnique`, `MakeReadUnique`	`CompData` beats from memory, `Resp` set to `UC_UD`, no RSP
Write	`WriteNoSnpFull`, `WriteNoSnpPtl`, `WriteBackFull`, `WriteBackPtl`, `WriteCleanFull`, `WriteUniqueFull`, `WriteUniquePtl`, `WriteEvictFull`, `WriteEvictOrEvict`, `WriteNoSnpDef`	`DBIDResp`, then the data beats are collected and applied, then `Comp`
Zero write	`WriteNoSnpZero`, `WriteUniqueZero`	`DBIDResp`, then `Comp`, no data expected
Everything else	including `Evict`	`Comp` only

Table 6.1 Request to response mapping

A Subordinate knows nothing about coherence, so the `UC_UD` it returns on a read is not a coherence decision. It is the Home's job to replace that field with the state the directory justifies, and the Home failing to do so was `iteration-log.md` item 18.

DBID and write-data matching. The DBID offered in `DBIDResp` is the request's own TxnID, and the write-data mailbox is keyed by `(SrcID, TxnID)`. Both matter: the Home relays the DBID it was given, so choosing an unrelated value breaks the return path, and keying the mailbox on TxnID alone collides when two sources use the same TxnID.

Completion ordering. `DBIDResp` and `Comp` are separate messages with the data phase between them. Sending a

combined `CompDBIDResp` up front completes the write before the data has arrived, which let the Home move on and misroute the data that followed: `iteration-log.md` item 10.

6.4 Monitor

Structurally the same as the Requester's, over REQ, RSP and DAT. A snoop flit observed at a Subordinate is an error: no protocol snoop reaches a Subordinate.

7 Checking

All checking is in `topScoreboard`. It subscribes to every requester and subordinate monitor through analysis FIFOs and consumes them in per-agent loops, so checks see flits in the order each agent saw them.

7.1 Reference models

Model	Storage	Key
Transaction table	`txnTable`	`r<agent>_t<txnid>`
Address by pending TxnID	`pendingAddr`	agent, then TxnID
DBID owner	`dbidOwner`	`r<agent>_p<peer>_d<dbid>`
Directory	`directory`	line address, then agent
Memory	`expectedLine`, `lineWritten`	line address
Writes in flight	`writesInFlight`	line address
Data beat assembly	`assembly`	`r<agent>_p<peer>_a<txnid>_<dir>`
Snoop line record	`snoopLineRecord`	`r<agent>_h<home>_s<txnid>`
Subordinate write address	`subWriteAddr`	`s<agent>_p<src>_w<txnid>`
Link state and RUN history	per agent, per direction	agent, direction

Table 7.1 Reference models and key schemes

The peer NodeID is part of the data, DBID and snoop keys for a reason. A Requester's own TxnID space and the TxnID space of the snoops sent to it are independent, and two Home Nodes may use the same TxnID towards one Requester. Keying on `(agent, TxnID)` alone merged a read's `CompData` with a concurrent `SnpRespData`: `iteration-log.md` item 11.

The memory model is maintained from both sides. Requester write data updates it, and so does write data observed arriving at a Subordinate, which is how writebacks that the Home originates on its own are folded in.

7.2 Check inventory

Check	Fails when
Requester receives a request	Any REQ flit arrives at a Requester
TxnID reuse	A request uses a TxnID already outstanding for that agent
`AllowRetry` discipline	A request has `AllowRetry` clear without a preceding `RetryAck`
`PCrdType` match	A credited resend carries a `PCrdType` other than the one granted
`RetryAck` target	A `RetryAck` names a TxnID that is not outstanding
Completion of a known transaction	A completion names an unknown transaction
Single completion	A transaction completes twice
Unique exclusivity	More than one Requester holds a line Unique
Unique versus shared	A line is held Unique while others also hold it
Snoopability	A Requester with no cacheable state is snooped
Write data direction	Write data is delivered to a Requester
DBID validity	Write data is sent on a DBID that was never offered
Read data integrity	A read data byte differs from the memory model
Subordinate never requests	A Subordinate sends a REQ flit
Address map	An SN-F receives an address the map routes to the HN-I region
Credit limit	Credits held on a channel exceed `lcrdDepth`
Unexpected error response	`DERR` or `NDERR` with no error injected
Class-legal opcodes	An I/O coherent Requester issues a request that requires a cache
Transfer size	`Size` exceeds a cache line

Table 7.2 Per-flit checks

A Requester snooped for a line the directory does not record it as holding is a warning rather than an error, because the model can legitimately lag the fabric by the flight time of a completion.

7.3 End-of-test checks

`check_phase` adds five checks that can only be made once the traffic has stopped:

Check	Fails when
Transmit link reached RUN	Any agent's transmit link never reached RUN
Receive link reached RUN	Any agent's receive link never reached RUN

Transactions completed	Any transaction is still open, naming its opcode, address and issue time
Data messages completed	Any beat assembly is partial, naming the packet count
Retry made progress	`RetryAck` responses were sent but no `PCrdGrant` ever followed

Table 7.3 End-of-test checks

The report phase prints a one-line summary of the traffic actually observed. For `chiCoherentTrafficTest` at seed 7:

```
scoreboard: 118 requests, 70 completions, 48 retries, 48 grants,

39 snoops; data checks 58 passed, 3 skipped for a concurrent write
```

Data checks are skipped, not failed, for a line that has a write in flight from another Requester, because the order of a concurrent read and write is not determined.

8 Coverage plan

8.1 Cross-agent coverage

`topCoverage` covers the properties that involve more than one agent, which is the reason it sits above the agents rather than inside one.

Covergroup	Coverpoints and bins	Crosses
`sharingCg`	Sharers per line: 0, 1, 2, 3, 4 or more. Unique holders: 0, 1, 2 or more (illegal)	sharers by unique holders
`classActivityCg`	Node class: RN-F, RN-D, RN-I. Request kind: read, write, upgrade, evict, other	class by kind
`retryCg`	`RetryAck` seen, grant seen, credited resend seen	full handshake

Table 8.1 `topCoverage`

The illegal bin on unique holders and the full-handshake cross are the two that matter: the first says the coherence invariant was actually observed rather than assumed, and the second says the retry flow was completed end to end and not merely started.

8.2 Requester coverage

Covergroup	Coverpoints	Crosses
`requestCg`	Opcode: `ReadShared`, `ReadUnique`, `ReadOnce`, `ReadNoSnp`, `CleanUnique`, `Evict`, `WriteBackFull`, `WriteUniqueFull`, `WriteNoSnpFull`, other. Node class. `AllowRetry`. Size: line, other	opcode by class, opcode by retry
`responseCg`	Opcode: `Comp`, `CompDBIDResp`, `DBIDResp`, `RetryAck`, `PCrdGrant`, `ReadReceipt`,	opcode by resp

	CompAck, SnpResp, other. Resp: I, SC, UC_UD, SD, pass-dirty. RespErr: OK, EXOK, DERR, NDERR	
snoopCg	Opcode: SnpShared, SnpClean, SnpOnce, SnpUnique, SnpCleanInvalid, SnpMakeInvalid, SnpNotSharedDirty, other. RetToSrc. DoNotGoToSD	opcode by RetToSrc
linkCg	Event kind, direction, channel, state	state by direction
dataCg	Opcode: CompData, CopyBackWrData, NonCopyBackWrData, SnpRespData, other. DataID. Direction	opcode by DataID

Table 8.2 chiRequesterCoverage

8.3 Subordinate coverage

Covergroup	Coverpoints	Crosses
downstreamRequestCg	Opcode: ReadNoSnp, ReadShared, ReadUnique, ReadOnce, WriteNoSnpFull, WriteNoSnpPtl, WriteBackFull, WriteUniqueFull, Evict, other. Node class: SN-F, SN-I. Size	opcode by class
responseCg	Opcode: Comp, CompDBIDResp, DBIDResp, other. RespErr	opcode by error
dataCg	Opcode: CompData, NonCopyBackWrData, CopyBackWrData, WriteDataCancel, other. DataID. Direction	opcode by DataID
linkCg	As the Requester's	state by direction

Table 8.3 chiSubordinateCoverage

8.4 Known coverage holes

The bins below exist and are reachable in principle but are not hit by the Stage 1 test list, because no stimulus produces them: the DEACTIVATE link state, EXOK, DERR and NDERR response errors, ReadReceipt, WriteDataCancel, partial-write data opcodes, the RN-D class bins, and the CHI_LINK_EVT_CREDIT_USE and CHI_LINK_EVT_CREDIT_ERROR event kinds, which the monitor never emits. They are left in place as the coverage target for the next stage rather than excluded.

9 Stimulus

9.1 Requester sequences

Sequence	Traffic	Knobs
chiRequesterBaseSeq	Base class	accessCount 8 by default, constrained to 1--64; addrBase 0x1000, addrRange 0x1000

`chiRequesterReadSeq`	Reads only	inherited
`chiRequesterWriteSeq`	Writes only	inherited
`chiRequesterRandomSeq`	Read and write mix	`writePercent` 35 by default, constrained to 20--50
`chiRequesterSharedLineSeq`	Four lines only, every fourth access a write	`addrRange` is four lines
`chiRequesterRetryStressSeq`	48 accesses over a 64 KiB window, every third a write	fixed

Table 9.1 Requester sequences

9.2 Subordinate policy sequences

Sequence	Policy	Knobs
`chiSubordinateBaseSeq`	Base class	`policyCount` 256 by default, constrained to 1--512
`chiSubordinateFixedLatencySeq`	Constant response delay, no errors	`latency` 2 by default, constrained to 0--8
`chiSubordinateRandomLatencySeq`	Random response delay 0--12, no errors	—

Table 9.2 Subordinate sequences

9.3 Virtual sequences

Sequence	What it runs
`chiVirtualBaseSeq`	Starts the Subordinate responders, fixed or random latency
`chiLinkBringUpVirtualSeq`	Responders only, no Requester traffic
`chiCoherentTrafficVirtualSeq`	`chiRequesterRandomSeq` on every Requester in parallel over a 2 KiB window from `0x2000`; `accessesPerAgent` 16 by default, constrained to 4--64
`chiSharedLineVirtualSeq`	`chiRequesterSharedLineSeq` on every Requester over four lines from `0x4000`, with random Subordinate latency; `accessesPerAgent` 24 by default
`chiRetryStressVirtualSeq`	`chiRequesterRetryStressSeq` on every Requester, with random Subordinate latency

Table 9.3 Virtual sequences

Running the same window from every Requester in parallel is what produces snoops: they arise from the traffic rather than from injected snoop stimulus, so the snoop is always the consequence of a real request the fabric chose to make.

10 Test plan

10.1 Test list

Test	Sequence	Overrides
`chiLinkBringUpTest`	`chiLinkBringUpVirtualSeq`	`drainCycles` 200
`chiCoherentTrafficTest`	`chiCoherentTrafficVirtualSeq`, 16 accesses per agent	none
`chiSharedLineTest`	`chiSharedLineVirtualSeq`, 24 accesses per agent	none
`chiRetryStressTest`	`chiRetryStressVirtualSeq`	`drainCycles` 4000; `maxOutstanding` 16 on every Requester
`chiPassiveAgentTest`	`chiCoherentTrafficVirtualSeq`, 8 accesses per agent	last Requester passive; scoreboard disabled

Table 10.1 Test list

The tests are ordered from least to most demanding on purpose. If the traffic tests fail, `chiLinkBringUpTest` says whether the links were ever the problem, and its failure has an unambiguous cause.

`chiRetryStressTest` needs the deeper outstanding limit to do its job. With four transactions per Requester the Home has enough resources and `RetryAck` is never sent; at sixteen the transaction pools are exhausted and the retry handshake is forced.

`chiPassiveAgentTest` disables the scoreboard rather than relaxing one check. A passive Requester has no driver, so nothing brings its link up and it stays in STOP, which the link check correctly reports as a failure. Checking is therefore left to coverage in this test, and the test's value is that the environment builds, connects and samples correctly when an agent does not drive.

10.2 Base test

`chiBaseTest` builds the configuration, calls `configureEnv()` for a derived test to override it, publishes it, and builds `topEnv`. In `run_phase` it raises an objection, forks a 2 ms timeout, runs `testSequence()` on the virtual sequencer, waits `drainCycles` and drops the objection.

`drainCycles` defaults to 2000. It exists because dropping the objection when the last sequence item is handed over would end the test with transactions still in flight: the end-of-test checks would then fire on transactions that were merely unfinished rather than stuck.

10.3 Regression status

All five tests pass with zero `UVM_ERROR` and zero `UVM_FATAL`. The traffic tests were additionally swept over 40 seeds each, 120 runs, all clean. Section 11.3 gives the regression command.

11 Run flow

11.1 Directory map

Path	Contents
`DV/src/globals/`	`chiGlobalsPkg`
`DV/src/hdlTop/`	Everything that touches a pin
`DV/src/hvlTop/chiEnv/`	Agents, virtual sequencer and virtual sequences
`DV/src/hvlTop/topEnv/`	Scoreboard, coverage and the top environment
`DV/src/hvlTop/test/`	Base test and the test library
`DV/sim/xcelium/`	Filelist, Makefile, probe script and the run area
`DV/doc/`	This document, the traceability table, the iteration log and the open questions

Table 11.1 DV directory map

11.2 Compile order

`chiFileList.f` is the single authoritative compile order. It is arranged in five sections, in this order: the DUT RTL, in the same order as the root `filelist.f` but without the lint-only compile harnesses that would elaborate extra copies of the fabric; the DUT assertion modules; the testbench globals package; the HDL side; and the HVL side, packages before tops. Include directories are listed ahead of the sources.

11.3 Make targets

Target	Action
`make compile`	Elaborate into a snapshot, log to `elab.log`
`make run`	Run `TEST` at `SEED` with `VERBOSITY`, log to `run_<TEST>.log`
`make regress`	Run all five tests and report PASS or FAIL per test
`make waves`	Run with `probe.tcl` and report the waveform database path
`make clean`	Remove the build and run artefacts

Table 11.2 Make targets

Variable	Default

TEST	chiCoherentTrafficTest
SEED	1
VERBOSITY	UVM_MEDIUM
ACCESS	+rwc

Table 11.3 Make variables

11.4 Waveforms

`probe.tcl` opens `waves.shm`, probes `hdlTop` to full depth including memories, and runs. `make waves` prints the database path rather than opening a viewer, so it works in a batch session.

11.5 Regenerating this document

```
cd DV/doc

make pdf
```

`build_pdf.py` renders the Markdown source to HTML and converts it with LibreOffice. The Markdown is the reviewable artefact; the PDF is build output.

12 Traceability

Every quantity the environment asserts is traceable to one of two places: the CHI specification, cited as `[SPEC:IHI0050H <section>]`, or the configuration document, cited as `[CFGDOC:<section>]`. Values taken from the RTL are cited as `[RTL:<path> <construct>]`, which marks them as observations of this implementation rather than requirements.

The rule is applied to signal names, field widths, node counts, opcode choices, state transitions and every check. `traceability.md` holds the table.

No quantity in the environment is a bare literal without a citation, with one class of exception: testbench-internal sizing that the architecture does not constrain, such as the 32-line cache model, the 64-entry TxnID scan and the drain and timeout values. Those are engineering choices and are named as such.

13 Known limitations

The environment's own limitations, as distinct from the fabric's:

1. **The cache model is not a cache controller.** It tracks state and data per line and answers snoops, but it has no write buffer, no coalescing and no partial-line handling, and every access is a full line.

1. **The directory model lags by design.** It is built from observed completions, so a snoop that arrives before the completion it races is reported as a warning rather than an error. A precise model would need to track the Home's ordering point, which is not observable at the node ports.

1. **Data checks are skipped under concurrency.** A read of a line with a write in flight from another Requester is not checked, because the order is not determined at the node ports. This is a real hole: a genuine coherence failure on exactly that line would be missed.

1. **One configuration only.** The default node counts are the only shape tested. The DUT's placement table prevents raising them without supplying placements explicitly.

1. **No error injection.** The hook exists in the Subordinate and is never used, so the fabric's error paths are unverified.

1. **Link deactivation and credit return are unexercised**, so the DEACTIVATE state and `LCrdReturn` handling in the fabric are unverified by this environment.

1. **The passive test does not check.** Disabling the scoreboard to accommodate a link that never comes up also disables every other check in that test.

Open questions about the fabric and the specification, as opposed to limitations of the environment, are in `open-questions.md`. The bring-up history, including the eleven RTL bugs this environment found, is in `iteration-log.md`.